

24.9cm X 25.4cm
冊絹设色
台北故宫博物院藏

《苹婆山鸟图》
黄筌 · 宋代

13:21
2026/11/2


No Signal


No Signal

PowerPoint File Edit View Insert Format Arrange Tools Slide Show Window Help 100% Mon Nov 2 13:29

AutoSave S3_Program_Verification — Saved to my Mac

Search (Cmd + Ctrl + U)

Home Insert Draw Design Transitions Animations Slide Show Record Review View

Comments Record Share

Play from Start | Play from Current Slide | Presenter View | Custom Show | Rehearse with Coach | Set Up Slide Show | Hide Slide | Rehearse Timings | Record | Keep Slides Updated | Use Timings | Play Narrations | Show Media Controls | Always Use Subtitles | Subtitle Settings

1

PROGRAM VERIFICATION

Proving Programs Correct

Program verification Basics

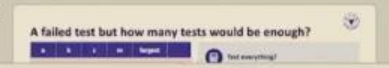
2



3



4



Where we are: three ways to trust code

Testing	Model checking	Program verification
covers	covers	covers
a few chosen inputs	all states of a small model	every input, any size
you get	you get	you get
pass / fail	✓ or a counterexample	✓ = a proof, or a counterexample

Today, on one tiny example: finding the largest number.

> **Program verification covers every input, like model checking, but by reasoning with unknown inputs instead of visiting states.**

GenAI-Empowered Industrial Programming · Program verification 2

Recap. Testing runs the code on a few chosen inputs: cheap, but it only sees those inputs. Model checking visits all reachable states of a small model automatically. Program verification reasons about the code with the inputs left unknown, like algebra, so one argument covers every input at once. (Under the hood a solver still searches, but for a failing input, not through states; we will see this.) The price: you must write down precisely what the code should do. Today, on a deliberately tiny example, so that the ideas are visible.

Slide 2 of 67 English (China) Accessibility: Investigate Notes Comments 123%

USB-C
IN : 1920X1080P 60
OUT : 3840X2160P 60

1

PROGRAM VERIFICATION

Proving Programs Correct

Program verification Basics

2

Where we are: three ways to trust code

3

An LLM wrote this. Four tests pass. Is it correct?

4

A failed test but how many tests would be enough?

Where we are: three ways to trust code

Testing
covers
a few chosen inputs
you get
pass / fail

→

Model checking
covers
all states of a small model
you get
✓ or a counterexample

→

Program verification
covers
every input, any size
you get
✓ = a proof, or a counterexample

Today, on one tiny example: finding the largest number.

> **Program verification covers every input, like model checking, but by reasoning with unknown inputs instead of visiting states.**

Recap. Testing runs the code on a few chosen inputs: cheap, but it only sees those inputs. Model checking visits all reachable states of a small model automatically. Program verification reasons about the code with the inputs left unknown, like algebra, so one argument covers every input at once. (Under the hood a solver still searches, but for a failing input, not through states; we will see this.) The price: you must write down precisely what the code should do. Today, on a deliberately tiny example, so that the ideas are visible.

Where we are: three ways to trust code



Testing

covers
a few chosen inputs

you get
pass / fail

→



Model checking

covers
all states of a small model

you get
✓ or a counterexample

→



Program verification

covers
every input, any size

you get
✓ = a proof, or a counterexample

Today, on one tiny example: finding the largest number.

> **Program verification covers every input, like model checking, but by reasoning with unknown inputs instead of visiting states.**

An LLM wrote this. Four tests pass. Is it correct?

the LLM's code: m = the largest of a, b, c

```
if a > b {
  if a > c { m := a; } else { m := c; }
} else {
  m := b;
}
```

a	b	c	m	largest	
3	2	1	3	3	✓
1	3	2	3	3	✓
2	1	3	3	3	✓
5	5	5	5	5	✓

a, b, c: whole numbers; m: the result

Vote Is the code correct for every input?

Code that passes every test you tried can still be wrong.

A failed test but how many tests would be enough?

a	b	c	m	largest	
1	2	3	2	3	✗

If a is not the largest, the code returns b without comparing b with c.



Test everything?

a 16-bit whole number: $2^{16} = 65{,}536$ values
three inputs: $65{,}536^3 \approx 2.8 \times 10^{14}$
at 1 µs per test: about 9 years

Even three small numbers have far too many combinations to test. We need a different idea.

The idea: reason about every input at once

Testing: plug in numbers

$(x + 1)^2 = x^2 + 2x + 1$?

$x = 1$: $4 = 4$ ✓
$x = 2$: $9 = 9$ ✓
$x = 3$: $16 = 16$ ✓
…and $x = 4$?

Proving: keep x unknown

$$(x + 1)^2 = (x + 1)(x + 1)$$
$$= x^2 + x + x + 1$$
$$= x^2 + 2x + 1 \quad ✓$$

one argument, every x

A program proof does the same: the inputs a, b, c stay unknown, and we follow the code.

> **A proof keeps the inputs unknown, so one argument covers all of them.**

Reading the symbols

in words	maths	Dafny	ST	example
and (both true)	∧	&&	AND	m ≥ a ∧ m ≥ b
or (at least one)	∨	||	OR	m = a ∨ m = b
not (the opposite)	¬	!	NOT	¬(m ≥ c) means m < c
is equal to	=	==	=	m == a
at least	≥	>=	>=	m >= a
gets the value (in code)		:=	:=	m := a

ST = Structured Text, the PLC language from Session 1: careful, ST's = compares, Dafny's == compares

Four spellings of the same few ideas: words, maths, Dafny, and the ST you know.

① Max of two numbers: from words to a contract

code: m = the larger of a and b

```
if a >= b {
  m := a;
} else {
  m := b;
}
```

M1 m is not smaller than a

$m \geq a$

M2 m is not smaller than b

$m \geq b$

M3 m is one of the two inputs

$m = a \quad \vee \quad m = b$

Ask Why do we need M3? What would m = 1000 do for a = 3, b = 5?

The contract states exactly what the code must guarantee after it runs, for every input.

① Proof step 1: what to assume, what to show

code

```
if a >= b {
  m := a;
} else {
  m := b;
}
```

We may assume

a and b are any whole numbers

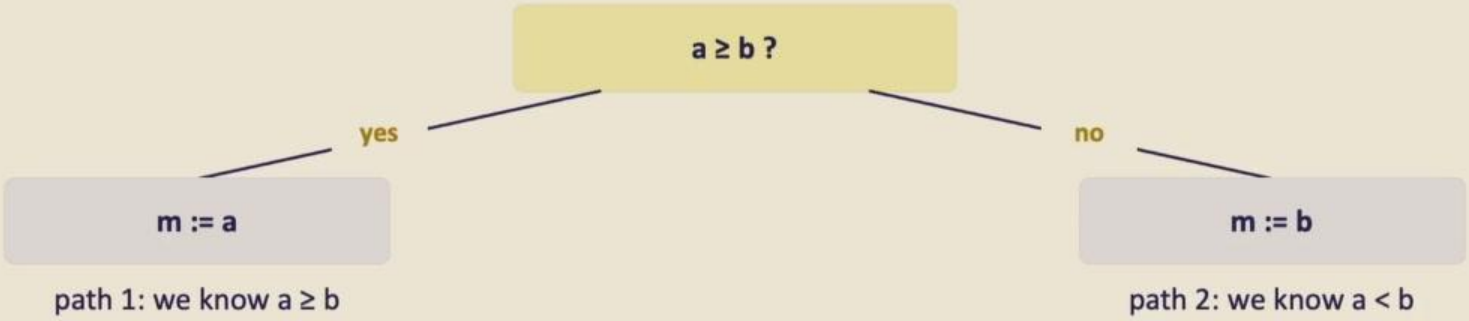
(nothing else: they stay unknown)

We must show

M1: $m \geq a$
M2: $m \geq b$
M3: $m = a \vee m = b$

A proof starts with two lists: what we may assume, and what we must show.

① Proof step 2: the IF splits the proof into two cases



Each IF condition is either true or false, so every input takes exactly one path, and on each path we learn something about the inputs.

An IF we cannot decide splits the proof into cases, one per branch.

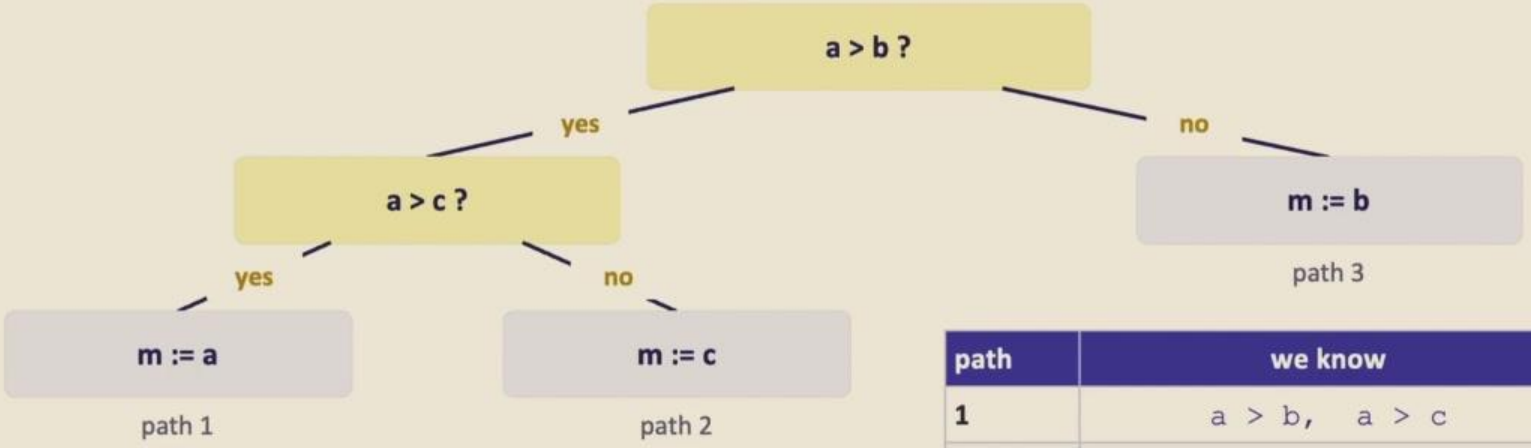
① Proof step 3: check M1, M2, M3 on each path

path	we know	m	M1: m ≥ a	M2: m ≥ b	M3: m = a ∨ m = b
1	a ≥ b	a	a ≥ a ✓	a ≥ b ✓ (we know)	m = a ✓
2	a < b	b	b ≥ a ✓ (we know)	b ≥ b ✓	m = b ✓

2 paths × 3 requirements, all ✓: Max is proven for every a and b

A proof = every requirement checked on every path. No input values were tried.

① The same steps on the LLM's Max3: three paths



path	we know	m
1	a > b, a > c	a
2	a > b, a ≤ c	c
3	a ≤ b (nothing about c!)	b

Three paths. On path 3 we know nothing about c: that is suspicious.

An LLM wrote this. Four tests pass. Is it correct?

the LLM's code: m = the largest of a, b, c

```
if a > b {
  if a > c { m := a; } else { m := c; }
} else {
  m := b;
}
```

a	b	c	m	largest	
3	2	1	3	3	✓
1	3	2	3	3	✓
2	1	3	3	3	✓
5	5	5	5	5	✓

a, b, c: whole numbers; m: the result

Vote Is the code correct for every input?

Code that passes every test you tried can still be wrong.

① Path 3 cannot be shown: a counterexample

path 3	we know	must show	result
`m = b`	`a ≤ b`	`m ≥ c, i.e. b ≥ c`	nothing known about c ✗

To break the code: take a ≤ b, and c larger than b.

counterexample: a = 1, b = 2, c = 3 → m = 2, but the largest is 3

A step that cannot be shown tells us how to break the code: that is a counterexample.

From "is it always true?" to "can it fail?"

always true?

hard to check directly: there are too many inputs

can it fail?

search for one input that breaks it:
impossible → always true ✓
found → counterexample ✗

Max, path 2, M1: `a < b (path) and m = b (code) and m < a (M1 false)`

needs b < a and a < b at the same time: impossible, so M1 always holds on path 2 ✓

> **"Always true" = "no input can make it false". Machines prove by searching for that input.**

How a program verifier works: our proof, automated



code + contract
you write

→



split into paths
proof step 2

→



ask a solver
can a requirement fail on this path?

impossible on every path → proven ✓

possible → a counterexample ✗

A program verifier does our hand proof automatically, for code far too big to do by hand.



Coq, Dafny, and how the machine proves

First a proof assistant that checks every step we write (Coq), then a verifier that finds the steps itself (Dafny), and a look inside its solver.

A proof assistant checks our hand proof



Coq (now Rocq)	
made by	INRIA, France; since 1989; renamed Rocq in 2025
famous proofs	CompCert C compiler; the four-colour theorem (2005)
relatives	Isabelle (seL4 kernel), Lean (maths, and AI provers later today)

In a proof assistant you write the proof; the machine checks every step of it.

① Proof step 3: check M1, M2, M3 on each path

path	we know	m	M1: m ≥ a	M2: m ≥ b	M3: m = a ∨ m = b
1	a ≥ b	a	a ≥ a ✓	a ≥ b ✓ (we know)	m = a ✓
2	a < b	b	b ≥ a ✓ (we know)	b ≥ b ✓	m = b ✓

2 paths × 3 requirements, all ✓: Max is proven for every a and b

A proof = every requirement checked on every path. No input values were tried.

Max in Coq: the program and the theorem

Max.v

```
Definition max (a b : Z) : Z :=
  if a >=? b then a else b.

Theorem max_correct : forall a b : Z,
  max a b >= a /\ max a b >= b /\
  (max a b = a \/ max a b = b).
```

Coq	read as
`Definition`	the code
`Theorem` … `forall a b`	the contract, for every a, b
`Z`	whole numbers (like Dafny's int)
`a >=? b`	is a ≥ b? gives true / false
`/\` `\/`	and (∧), or (∨)

M1: `max a b >= a` M2: `max a b >= b` M3: `max a b = a \/ max a b = b`

The contract becomes a theorem: a precise statement about the code, for every input.

Proving step by step: the proof state

you type

```
Proof.
  intros a b.
  unfold max.
```

Tactic

intros: a, b are any numbers
unfold: look inside the code

Coq shows: after intros a b

```
a, b : Z
```

we may assume ▲

---

we must show ▼

```
max a b >= a /\ max a b >= b /\ (...)
```

after unfold max: the IF is visible

```
a, b : Z
```

we may assume ▲

---

we must show ▼

```
(if a >=? b then a else b) >= a /\
(if a >=? b then a else b) >= b /\ (...)
```

Above the line: what we may assume. Below: what we must show. Each tactic changes the picture.

Splitting on the IF: one goal per path

```
destruct (Z.geb_spec a b) as [H | H].
```

goal 1: path 1 (the test was true)

```
a, b : Z
H : b <= a
```

```
a >= a /\ a >= b /\
(a = a \/ a = b)
```

goal 2: path 2 (the test was false)

```
a, b : Z
H : a < b
```

```
b >= a /\ b >= b /\
(b = a \/ b = b)
```

the IF has disappeared: on each path Coq replaced it by the branch taken, and H records what we know

destruct is proof step 2: one goal per path, each with what we know on that path.

Finishing the proof: by hand, or with lia

Max.v (proof)

```
Proof.
  intros a b.
  unfold max.
  destruct (Z.geb_spec a b) as [H | H].
  - (* path 1: H : b <= a *)
    split. lia.            (* M1 *)
    split. lia.            (* M2 *)
    left. reflexivity.     (* M3 *)
  - (* path 2: H : a < b *)
    lia.                   (* all three *)
Qed.
```

tactic	step of the hand proof
`intros`	a, b stay unknown (step 1)
`unfold`	look inside the code
`destruct`	split on the IF (step 2)
`split, left`	one part of AND / OR
`reflexivity`	x = x
`lia`	arithmetic: a small solver
`Qed`	the kernel checks it all

coqc Max.v: no errors

Every step of the hand proof becomes a tactic; Coq accepts the proof only if every step is valid.

The LLM's Max3 in Coq: the proof gets stuck

Max3LLM.v (abridged)

```
Definition max3 (a b c : Z) : Z :=
  if a >? b then (if a >? c then a else c)
  else b.
...
  destruct (Z.gtb_spec a b) as [H1 | H1].
  - destruct (Z.gtb_spec a c) as [H2 | H2]; lia.
  - lia.      (* path 3 *)
```

Coq says

```
File "./Max3LLM.v", line 16:
Error: Tactic failure:
  Cannot find witness.
```

the goal left open (path 3)

```
H1 : a <= b
______________________________________
b >= a /\ b >= b /\ b >= c
```

Coq shows where the proof is stuck (b ≥ c, knowing only a ≤ b), but no input that breaks the code ("witness" = lia's word for a proof, not a counterexample)

A stuck proof says "not proved here", not "here is the bug": you look for the counterexample yourself.

The price of a proof assistant: proof and code live apart

seL4 OS kernel (Isabelle, 2009)



≈ 23 lines of proof per line of code
about 25 person-years (design, code and proof)
our Max in Coq: 2 lines of code, ~10 lines of proof

What an engineer wants

✓ write in a normal programming language
✓ the requirement right next to the code
✓ the machine does the proof steps
✓ a counterexample when the code is wrong
✓ ship exactly the code that was checked

Proof assistants give the strongest guarantee, but the proof lives apart from the code and costs far more than it.

Dafny's idea: the proof is part of the program

MaxArray.dfy: one file, one language

```
method MaxArray(a: array<int16>) returns (m: int16)
  requires a.Length > 0
  ensures forall j :: 0 <= j < a.Length ==> a[j] <= m
  ensures exists j :: 0 <= j < a.Length && a[j] == m
{
  m := a[0];
  var i := 1;
  while i < a.Length
    invariant 1 <= i <= a.Length
    invariant forall j :: 0 <= j < i ==> a[j] <= m
    invariant exists j :: 0 <= j < i && a[j] == m
  {
    if a[i] > m { m := a[i]; }
    i := i + 1;
  }
}
```

code 10 lines
ordinary statements: if, while, arrays, methods, like C# or Java

contract 3 lines
what it must do: requires, ensures

proof hints 3 lines
only where the solver needs help: invariant, assert, lemma

proof steps 0 lines: the solver
case splits, arithmetic, … (Coq: by hand)

same file → change the code, the proof is re-checked at once
same code → what was verified is what is compiled and shipped

Dafny: code, contract and proof hints in one program; the solver writes the proof steps.

From Coq to Dafny: who writes the proof steps?

	Coq: proof assistant	Dafny: automatic verifier
you write	code + theorem + every proof step	code + contract (+ loop invariants)
the machine	checks each step (a small kernel)	finds the steps itself (Z3 solver)
if it fails	a goal it cannot close	the failing line + a counterexample
running code	extract to OCaml, Haskell	compile to C#, Java, Python, ...
best for	any maths: compilers, OS kernels, theorems	everyday code: numbers, arrays, loops
Max	**about 10 lines of proof**	**0 lines: the solver does it**

> **Dafny keeps the idea, a contract and a proof, but lets a solver write the proof steps.**

Meet Dafny



you write
code + contract in one .dfy file

→



Dafny verifies
on every edit:
green ✓ or red ✗

→



Dafny compiles
to C#, Java, JavaScript, Go, Python

when	what
2009	designed by Rustan Leino, Microsoft Research; free and open source
2017 →	developed at Amazon Web Services
2024	AWS cloud access engine (10^9 calls per second) proved in Dafny

> **Dafny: a programming language whose compiler also proves.**

Dafny in industry: verified code that ships

AWS authorization engine (2024)	checks every AWS request, ~10^9 per second; proved equal to the old engine, compiled to Java, 3× faster
AWS Encryption SDK	the crypto library customers use: written in Dafny, compiled to .NET, Rust, …
Cedar policy language (AWS, 2023)	key properties proved on a Dafny model; ~10^8 tests a night check the Rust code against it
IronFleet (Microsoft Research, 2015)	distributed systems: Paxos replication and a key-value store, proved correct
Ethereum deposit contract (ConsenSys, 2021)	the Merkle-tree algorithm of the staking deposit contract, verified
LLMs (2024 →)	DafnyBench, vericoding: LLMs write code + hints, Dafny checks

common pattern: logic-heavy code, run millions of times, where one bug is a security incident

Dafny is used where a bug is too expensive, and the verified code is the code that runs.

Reading Dafny: Max with its contract

Dafny (Max.dfy)

```
method Max(a: int16, b: int16) returns (m: int16)
  ensures m >= a                // M1
  ensures m >= b                // M2
  ensures m == a || m == b      // M3
{
  if a >= b {
    m := a;
  } else {
    m := b;
  }
}
```

`method … returns (m)`
inputs a, b; output m

`int16`
a 16-bit whole number

`ensures`
a line of the contract

`// …`
a comment

`:=  ==  >=  ||`
see the symbol dictionary

Dafny: verified, 0 errors

The contract sits right above the code, in the same file.

Verify while you type

Max3.dfy — VS Code with the Dafny extension (illustration)

```
1    method Max3(a: int16, b: int16, c: int16) returns (m: int16)
2 ✗    ensures m >= a && m >= b && m >= c
3 ✓    ensures m == a || m == b || m == c
4    {
5      if a > b {
6        if a > c { m := a; } else { m := c; }
7      } else {
8        m := b;
9      }
10   }
```

```
Error: a postcondition could not be proved
  ensures m >= a && m >= b && m >= c
counterexample: a = 0, b = 0, c = 1, m = 0
```

fix the code → the red line disappears

The bug is underlined seconds after it was typed, with a counterexample.

Dafny's answer for the LLM's Max3

Dafny says (abridged)

```
Max3LLM.dfy(7,0): Error: a postcondition could not be
  proved on this return path
 Related counterexample:
  a = 0, b = 0, c = 1,  m = 0
Max3LLM.dfy(5,32): Related location: this is the
  postcondition that could not be proved
5 | ensures m >= a && m >= b && m >= c
                               ^^^^^^
Dafny program verifier finished with
  1 verified, 1 error
```

postcondition

= an ensures line: what must hold after the code

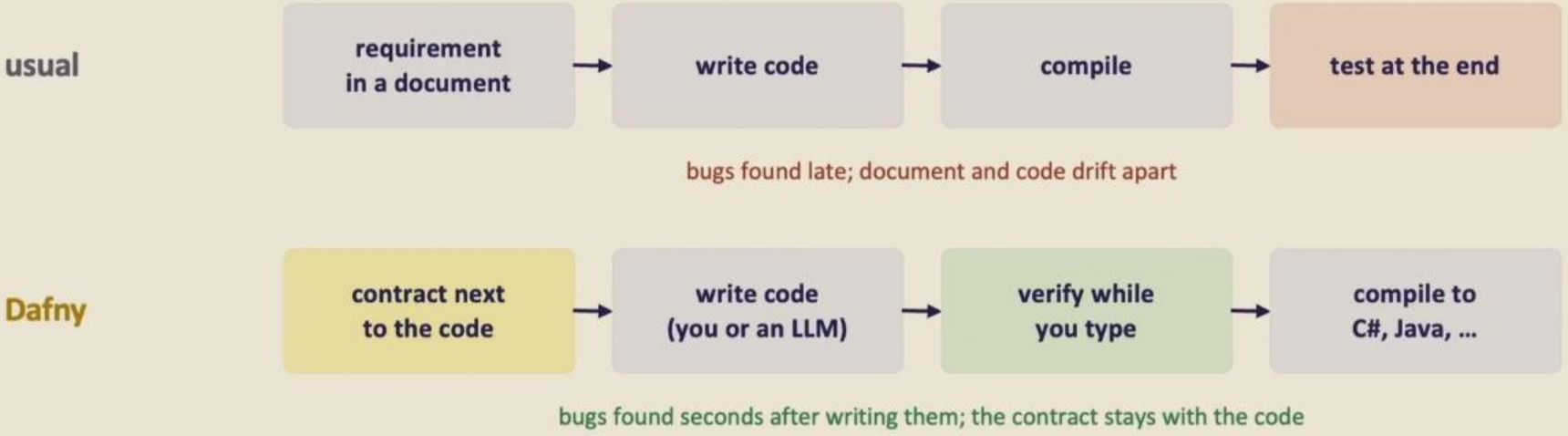
"could not be proved"

either the code is wrong (here),
or Dafny is missing information (block ③)

`(7,0)` = line 7, column 0. **0, 0, 1 instead of our 1, 2, 3:** any a ≤ b < c breaks the code; the solver just picks one.

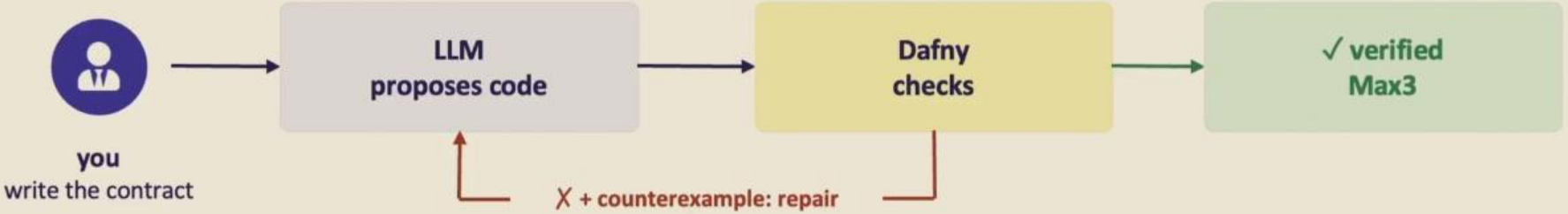
Dafny names the broken line of the contract and inputs that break it.

The usual way vs. the Dafny way



Dafny moves checking from the end of the project to the moment you type.

Propose, check, repair: an LLM with a checker



the repaired Max3: b is now compared with c

```
if a > b {
  if a > c { m := a; } else { m := c; }
} else {
  if b > c { m := b; } else { m := c; }
}
```

Dafny: 2 verified, 0 errors

you own the contract; the checker decides

The LLM may guess wrong many times; the checker never lets a wrong version through.

Inside the solver: one question per path

$$\neg(a > b) \quad \wedge \quad m = b \quad \wedge \quad \neg(m \geq c) \qquad ?$$

path 3 — the code — the requirement is false

Max3's requirement has three parts (m ≥ a, m ≥ b, m ≥ c); here we ask about one: m ≥ c. All three at once: idea 2.



word	meaning
satisfiable	values exist that make the formula TRUE
SAT solver	finds such values for TRUE / FALSE variables
SMT solver	the same, also with numbers: + − < ≥ (Z3, used by Dafny)

A solver answers one question: can this formula be made TRUE? Values, or "impossible".

Solver idea 1: write in what is forced

$$a < b \quad \wedge \quad m = b \quad \wedge \quad \neg(m \geq a) \qquad ?$$

Max, path 2, M1 false

step	forced by	we now know
1	the part $m = b$	replace m by b
2	the part $\neg(m \geq a)$	$\neg(b \geq a)$, i.e. $b < a$
3	the part $a < b$	✗ **contradiction with step 2**

impossible: M1 holds on path 2 ✓

Like the obvious cells of a Sudoku.

Propagation: write in everything that is forced. A contradiction means: impossible.

Idea 1 on the LLM's Max3: no contradiction

$$\neg(a > b) \quad \wedge \quad m = b \quad \wedge \quad \neg(m \geq c) \qquad ?$$

step	forced by	we now know
1	the part $m = b$	replace m by b
2	the part $\neg(m \geq c)$	$c > b$
3	the part $\neg(a > b)$	$a \leq b$
	anything contradicting?	**no: pick numbers**

values exist: a = 0, b = 0, c = 1 (Dafny's counterexample)

No contradiction: the solver picks numbers, and they form the counterexample.

Solver idea 2: nothing forced? Guess, and go back

$$a > b \;\wedge\; a > c \;\wedge\; m = a \;\wedge\; (m < a \;\vee\; m < b \;\vee\; m < c) \quad ?$$

the LLM's Max3, path 1 — all three parts of the requirement false: at least one fails

propagate m = a: $a < a \;\vee\; a < b \;\vee\; a < c$ **an OR: nothing is forced**

guess 1: a < a	✗ dead end: never true	go back, try the next
guess 2: a < b	✗ dead end: but the path says a > b	go back, try the next
guess 3: a < c	✗ dead end: but the path says a > c	no options left

every guess is a dead end → impossible: path 1 proven ✓

DPLL (1960–62): like a maze

When nothing is forced, guess; at a dead end, go back and try the next option.

Solver idea 3: learn from every dead end

without learning

branch 1	✗ a > b and a < b
branch 2	✗ a > b and a < b (again)
branch 3	✗ a > b and a < b (again)
branch 4	✗ a > b and a < b (again)

the same dead end, searched four times

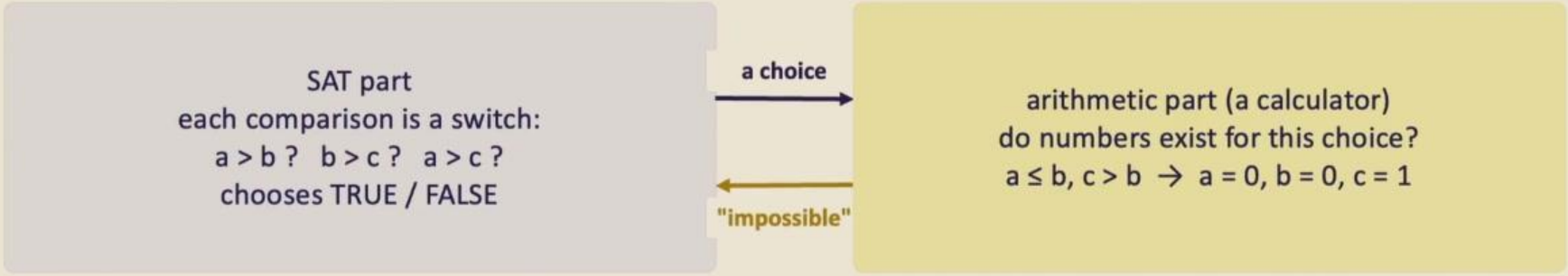
with learning

branch 1	✗ a > b and a < b
branch 2	cut at once by the rule
branch 3	cut at once by the rule
branch 4	cut at once by the rule

new rule after branch 1: "a > b and a < b: never together"

Learning (CDCL, 1996–2001): turn each dead end into a rule, so it is never searched again.

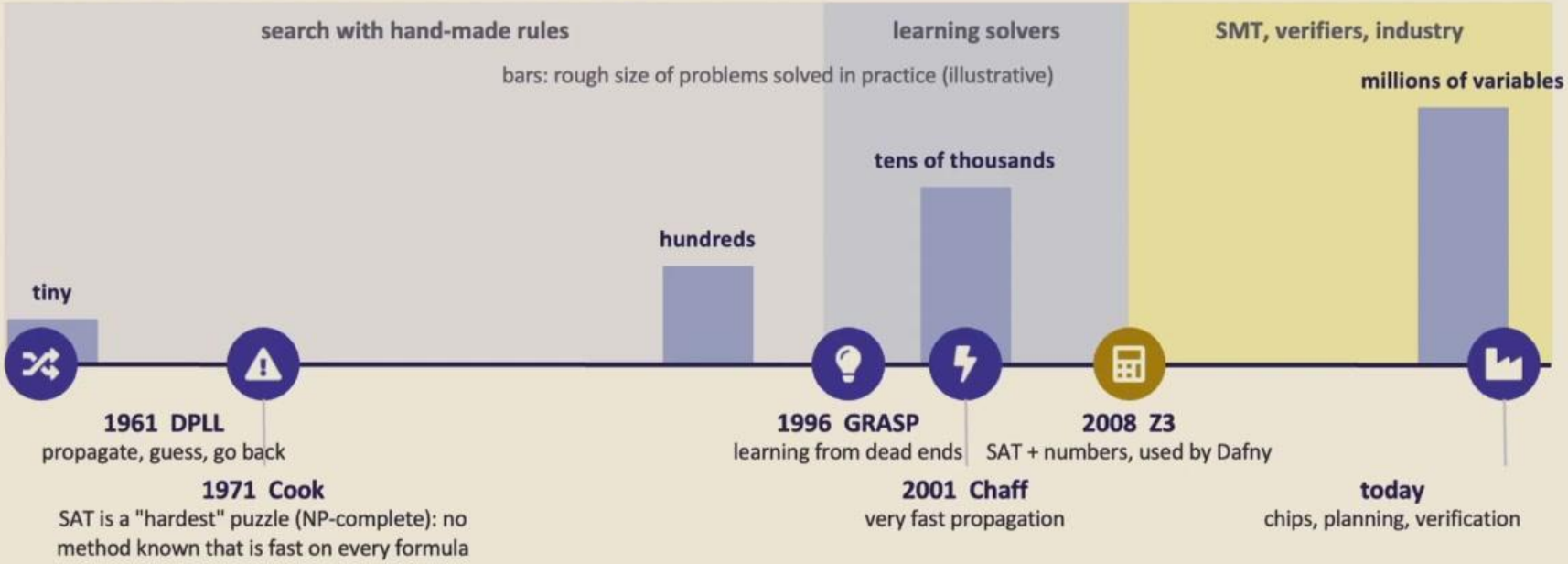
Solver idea 4: SMT = switches + a calculator



Z3 (Microsoft Research, 2008) is an SMT solver; Dafny sends every question to Z3.

SMT = a SAT solver for the TRUE/FALSE structure + a calculator for the numbers.

Sixty years of solver progress



Hard in theory, fast in practice: real problems have structure, and solvers learn it.

Finishing the proof: by hand, or with lia

Max.v (proof)

```
Proof.
  intros a b.
  unfold max.
  destruct (Z.geb_spec a b) as [H | H].
  - (* path 1: H : b <= a *)
    split. lia.            (* M1 *)
    split. lia.            (* M2 *)
    left. reflexivity.     (* M3 *)
  - (* path 2: H : a < b *)
    lia.                   (* all three *)
Qed.
```

tactic	step of the hand proof
`intros`	a, b stay unknown (step 1)
`unfold`	look inside the code
`destruct`	split on the IF (step 2)
`split, left`	one part of AND / OR
`reflexivity`	x = x
`lia`	arithmetic: a small solver
`Qed`	the kernel checks it all

coqc Max.v: no errors

Every step of the hand proof becomes a tactic; Coq accepts the proof only if every step is valid.

Splitting on the IF: one goal per path

```
destruct (Z.geb_spec a b) as [H | H].
```

goal 1: path 1 (the test was true)

```
a, b : Z
H : b <= a
```

```
a >= a /\ a >= b /\
(a = a \/ a = b)
```

goal 2: path 2 (the test was false)

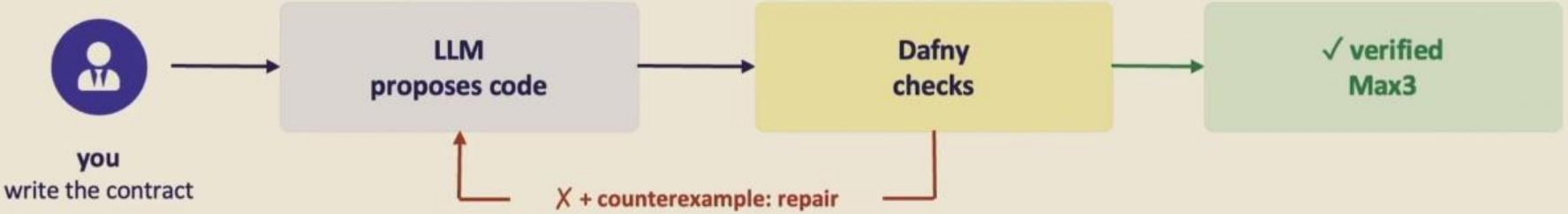
```
a, b : Z
H : a < b
```

```
b >= a /\ b >= b /\
(b = a \/ b = b)
```

the IF has disappeared: on each path Coq replaced it by the branch taken, and H records what we know

destruct is proof step 2: one goal per path, each with what we know on that path.

Propose, check, repair: an LLM with a checker



the repaired Max3: b is now compared with c

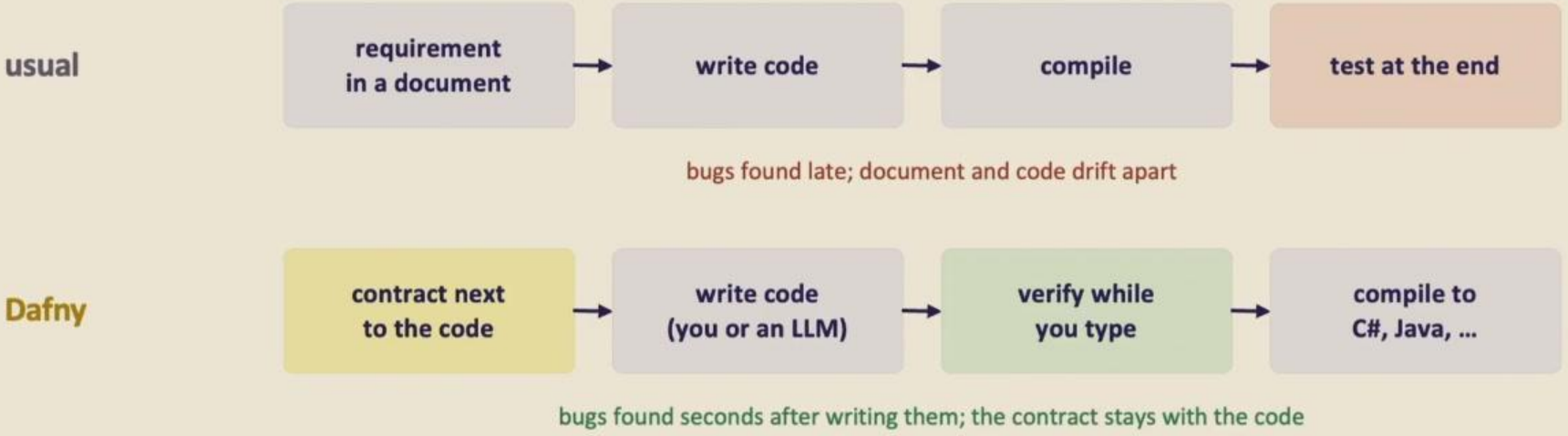
```
if a > b {
  if a > c { m := a; } else { m := c; }
} else {
  if b > c { m := b; } else { m := c; }
}
```

Dafny: 2 verified, 0 errors

you own the contract; the checker decides

The LLM may guess wrong many times; the checker never lets a wrong version through.

The usual way vs. the Dafny way



Dafny moves checking from the end of the project to the moment you type.

Solver idea 1: write in what is forced

$$a < b \quad \wedge \quad m = b \quad \wedge \quad \neg(m \geq a) \qquad ?$$

Max, path 2, M1 false

step	forced by	we now know
1	the part $m = b$	replace m by b
2	the part $\neg(m \geq a)$	$\neg(b \geq a)$, i.e. $b < a$
3	the part $a < b$	✗ **contradiction with step 2**

impossible: M1 holds on path 2 ✓

Like the obvious cells of a Sudoku.

Propagation: write in everything that is forced. A contradiction means: impossible.



Numbers, loops, limits, and AI

When a computer's numbers are not the integers, how to prove a loop, what a proof does not cover, and how AI proves.

③ The average of two numbers: the maths is right

Dafny with ideal whole numbers (int)

```
method AverageMath(a: int, b: int) returns (m: int)
  requires a <= b
  ensures  a <= m <= b
{
  m := (a + b) / 2;
}
```

Dafny: verified

`int`

a whole number without limits

`requires a <= b`

we may assume a ≤ b (like proof step 1; more in ④)

`a <= m <= b`

a ≤ m and m ≤ b

Ask Would you use m := (a + b) / 2 in your program?

With ideal whole numbers, the average is always between a and b.

③ …but a computer has only 16 (or 32, 64) bits

Dafny with 16-bit numbers (Average.dfy)

```
method Average(a: int16, b: int16) returns (m: int16)
  requires a <= b
  ensures  a <= m <= b
{
  m := (a + b) / 2;
}
```

Dafny says (abridged)

```
Average.dfy(17,10): Error:
 result of operation might
 violate newtype constraint
 for 'int16'
 counterexample:
  a = 16384, b = 16384
 m := (a + b) / 2;
       ^^^^^
```

Dafny's words "might violate newtype constraint" = the result might not fit in int16

int16: 2^{16} = 65,536 values, half of them negative: **`-32768`** … **`32767`**

`16384 + 16384 = 32768` : one more than the largest, 32767.

Both inputs are fine; only the sum a + b does not fit. Dafny finds it from the data type alone.

③ What happens when a number does not fit

Like an odometer

99999 + 1 → 00000

16 bits:
32767 + 1 → −32768

a = b = 30000	value
a + b (maths)	60000
stored in 16 bits	60000 − 65536 = −5536
÷ 2	−2768

the average of 30000 and 30000 comes out as −2768, without any error message

Too large numbers wrap around silently: a tiny formula, a completely wrong result.

③ A real bug for nine years, and its verified fix



Java's binary search, 2006

binary search: find a value in a sorted list by looking at the middle, then the middle of the right half, … (like a dictionary)

middle := (low + high) / 2 (low, high: positions)

wrong for ~9 years in Java's library: for lists of ~a billion (2^{30}) elements, low + high exceeds 32 bits

the fix (AverageFixed.dfy)

```
method Average(a: int16, b: int16) returns (m: int16)
  requires 0 <= a <= b    // positions
  ensures  a <= m <= b
{
  m := a + (b - a) / 2;
}
```

Dafny: verified, 0 errors

Give the program its real number types, and the proof covers what tests miss.

③ Why the fix cannot overflow, step by step

step	value lies between	fits in 16 bits?
`we may assume`	`0 ≤ a ≤ b ≤ 32767`	✓
`b - a`	`0 … b`	✓
`(b - a) / 2`	`0 … b - a`	✓
`a + (b - a) / 2`	`a … a + (b - a) = b`	✓ and the contract a ≤ m ≤ b

without 0 ≤ a, Dafny finds: a = −32768, b = 0 → b − a = 32768: one too large ✗

Every intermediate value stays between 0 and b, so nothing overflows. Dafny checks exactly these steps.

④ The largest number in a list: requires



Like a vending machine

if you insert enough coins
(your promise: requires),
you get your drink
(its guarantee: ensures)

Dafny

```
method MaxArray(a: array<int16>)
                returns (m: int16)
  requires a.Length > 0   // not empty
  ensures  …              // next slide
```

without requires: "index out of range" for an empty list

a[0] = the first element (positions count from 0); an empty list has no a[0]

	who must make it true	here
`requires`	the caller	the list is not empty
`ensures`	the function	m is the largest

requires = the caller's promise, ensures = the function's guarantee. Together: a contract.

④ The loop, by hand

the loop

```
m := a[0];
var i := 1;
while i < a.Length {
  if a[i] > m {
    m := a[i];
  }
  i := i + 1;
}
```

after round	start	1	2	3	4
element	a[0] = 4	9	2	11	7
m	**4**	**9**	**9**	**11**	**11**

Ask After each round, what is always true about m and the elements seen so far?

④ Reading the contract: "for all" and "if … then"

the contract of MaxArray

```
ensures forall j :: 0 <= j < a.Length ==> a[j] <= m
ensures exists j :: 0 <= j < a.Length && a[j] == m
```

Dafny	read as
`forall j :: …`	for every position j: …
`exists j :: …`	for at least one position j: …
`0 <= j < a.Length`	j is a position of the list (0 … n − 1)
`P ==> Q`	if P then Q

rain?	umbrella?	promise kept?
yes	yes	✓
yes	no	✗
no	yes	✓
no	no	✓

m is at least every element, and m is one of them. "If … then" fails only when the if is true and the then false.

④ Induction: the domino principle



...

base: the first one falls

true before the first round

step: each pushes the next

if true before a round, true after it

Base + step ⇒ all dominoes fall, however many: true after any number of rounds.

Induction checks two things instead of every round.

④ The loop invariant: m ≥ everything seen so far

invariant: m ≥ every element seen so far, and m is one of them

	why it holds
base (before the loop)	only a[0] seen, and m = a[0]
step (one round)	larger element → m takes it; otherwise m is already larger
end (i = length)	all elements seen → m ≥ every element: the contract ✓

A loop invariant is a domino statement: true at the start and kept by every round.

④ How Dafny checks a loop: three checks

1 before the loop
the invariant holds at the start (base)

→

2 one round
invariant before any round ⇒ invariant after it (step)

→

3 after the loop
invariant + loop finished ⇒ the contract

In check 2, Dafny knows only the invariant and i < a.Length. Everything else about m and i is forgotten: the round could be the 1st or the 1000th. So all the proof needs must be in the invariant.

+ Dafny also checks that the loop ends: a.Length − i shrinks every round (it finds this by itself).

One arbitrary round stands for all rounds: that is why the invariant must be written down.

④ In Dafny: the largest number, for any length

Dafny (MaxArray.dfy)

```
method MaxArray(a: array<int16>) returns (m: int16)
  requires a.Length > 0
  ensures forall j :: 0 <= j < a.Length ==> a[j] <= m
  ensures exists j :: 0 <= j < a.Length && a[j] == m
{
  m := a[0];
  var i := 1;
  while i < a.Length
    invariant 1 <= i <= a.Length
    invariant forall j :: 0 <= j < i ==> a[j] <= m
    invariant exists j :: 0 <= j < i && a[j] == m
  {
    if a[i] > m { m := a[i]; }
    i := i + 1;
  }
}
```

verified, 0 errors

reading the invariants

`0 <= j < i`

the positions seen so far

`1 <= i <= a.Length`

at the end i = a.Length: all were seen

without the invariant:
"could not be proved"
code right, information missing

Three invariant lines, and the result holds for lists of 2, 10 or a million numbers.

What a proof does not guarantee

the contract itself	wrong or missing requirements are proved anyway (Max without M3)
a hand translation	Dafny rewritten by hand in another language is not verified
what runs below	compiler, operating system, hardware
time and memory	right result, but maybe too slow

A proof shows the code keeps its contract. Whether the contract is right is your job.

Where proofs are used today

seL4 OS kernel (2009)	the C code matches its specification	**Isabelle/HOL**
CompCert C compiler	compiling never changes a program's meaning	**Coq / Rocq**
Intel floating-point units	arithmetic is correct (after the 1994 FDIV bug)	**HOL Light**
Paris Métro line 14 (1998)	driverless train control software	**B method**
Amazon Web Services (2024)	cloud access engine, 10^9 calls per second	**Dafny**

Where a bug is too expensive (chips, rail, aviation, cloud), people prove.

From verified Dafny to running code



① compile
to C#, Java, JavaScript, Go, Python: no hand rewriting

→



② generated tests
one test per path, run on code in another language

→



③ reference model
rewrite by hand (e.g. ST on a PLC), review against Dafny

① keeps the full guarantee; ② and ③ carry part of it to code that Dafny cannot check.

Verified Dafny becomes the program itself, a test source, or a reviewed reference.

Generated tests: Dafny writes one test per path

dafny generate-tests Block Max (abridged)

```
method {:test} Test0() {
  var r0 := Example.Max(-1, 0);      // path 2
  expect r0 >= -1;
  expect r0 >= 0;
  expect r0 == -1 || r0 == 0;
}
method {:test} Test1() {
  var r0 := Example.Max(0, 0);       // path 1
  expect r0 >= 0;
  ...
```

one test per path

(−1, 0): a < b, path 2
(0, 0): a ≥ b, path 1

expect

a check when the test runs: the contract, turned into a test

`Block` = one test per block of code, i.e. per path. `Example.Max` = the method Max in the module Example.

The proof's paths become tests that any other implementation must pass.

Verification tools: who writes the proof?

family	you write	the tool does	examples
model checker	model + property	searches all states	NuSMV, CBMC
verifier like Dafny	code + contract + invariants	finds the proof with a solver	Dafny, SPARK, Frama-C, Verus
proof assistant	the proof, step by step	checks every step	Isabelle, Rocq (Coq), Lean

◀ more automatic, smaller problems more effort, any mathematics ▶

Ask Which family did we use in the last session? Which two today?

Back to PLCs: the same ideas on industrial code

today, with Dafny	in PLC practice
contract: requires / ensures	formal specification of the requirement: LTL, patterns, assertions
verifier (Dafny + Z3)	PLCverif, NuSMV / nuXmv: model checkers and solvers underneath
counterexample: inputs that break it	a sequence of PLC scans, replayable in a simulator or on the PLC
propose, check, repair (LLM + Dafny)	Agents4PLC: LLM agents + compiler + verifier in a closed loop
what a proof does not guarantee	the specification itself, timers and hardware, the real I/O

Why not Dafny on the PLC code? Dafny does not read ST, and PLC requirements talk about sequences of scans ("after an E-Stop ..."): model checkers are built for that. In Dafny terms, one scan = one method call.

Same ideas, industrial tools: specify, verify, get a counterexample, repair.

The verified PLC pipeline, on the tank from the opening



Tank: "After an E-Stop, the filling must not restart by itself" → the LLM's FB_Fill → the verifier finds the 2-scan restart → repaired.

Two steps need new skills: writing the specification, and running the verifier.

Step 1: from a sentence to a formal specification

form	R2 of the tank
in words	When the E-Stop is pressed, the valve closes immediately.
precise	At the end of every scan in which estop is TRUE, valve is FALSE.
LTL formula	`G (estop → ¬valve)` `(G = always)`
pattern (PLCverif)	If estop is true at the end of the PLC cycle, then NOT valve should always be true at the end of the same cycle.
assertion in the code	`//#ASSERT NOT (estop AND valve)` `(at the end of the scan)`
as today, in Dafny	`ensures estop ==> !valve` `(one scan = one method call)`

names: estop = E-Stop pressed (opening: NOT estopOK) · stop = Stop pressed (NOT stopOK) · full = tank full (lsh) · assertion line: illustrative syntax

First make the words precise; then any of these forms says the same thing to a tool.

The hard part: deciding what the words mean

R3: "After an E-Stop, the filling must not restart by itself."

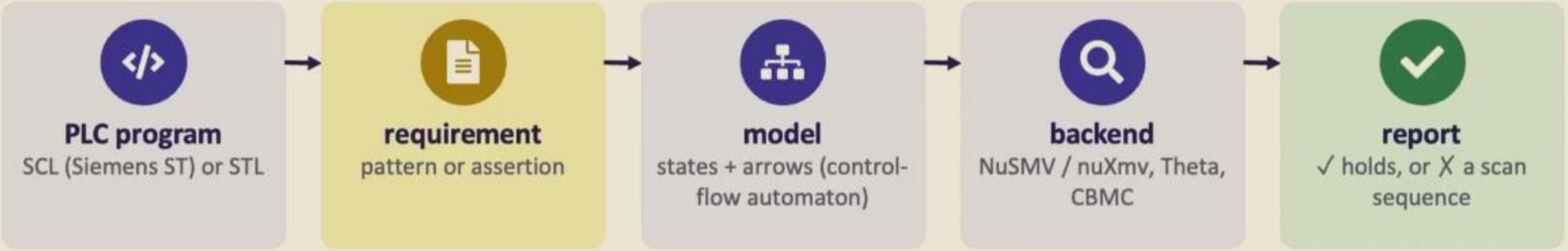
question	decision
what is "restart"?	the valve is closed in one scan and open in the next
what is "by itself"?	in a scan without a new Start press
what is a "new press"?	start is TRUE now and was FALSE in the last scan (rising edge)
and "after an E-Stop"?	dropped: never a restart without a new press, E-Stop or not (stricter, simpler)

```
(!valvePrev & valve) -> (start & !startPrev)
```
NuSMV invariant, opening

`valvePrev, startPrev`: extra variables, the value in the previous scan `!` NOT `&` AND `->` if ... then

Writing the specification forces engineering decisions. An LLM may draft it; an engineer must review it.

Step 2: how PLCverif checks PLC code



	PLCverif
made by	CERN, for its own PLC code, including safety systems
availability	open source since 2020 (gitlab.com/plcverif-oss)
underneath	the model checkers of Session 2 and the solvers of today

PLCverif turns PLC code + requirement into a model, and lets a model checker or solver decide.

What the verifier gives back: a counterexample

scan	start	stop	estop	full	filling	valve	what happens
1	T	F	T	F	T	F	Start during the E-Stop: request set, valve closed
2	F	F	F	F	T	T	**release all: the valve opens, no new Start ✗**

names: estop = E-Stop pressed (opening: NOT estopOK) · stop = Stop pressed (NOT stopOK) · full = tank full (lsh)

the LLM's FB_Fill (names as above)

```
IF start THEN filling := TRUE; END_IF;
IF stop OR full THEN filling := FALSE; END_IF;
valve := filling AND NOT estop AND NOT full;
```

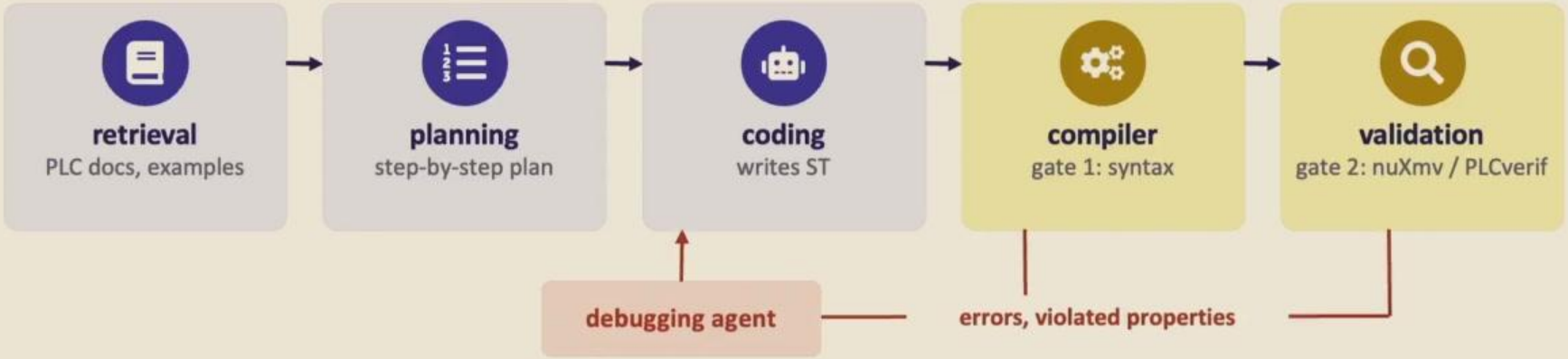
why it fails

filling = the fill request, remembered between scans
scan 1: Start sets it; the E-Stop never clears it
scan 2: E-Stop released → the old request opens the valve

then: replay it in a simulator or on the PLC · keep it as a regression test · feed it back to whoever repairs the code

A counterexample is a concrete scan sequence: it shows the cause, and becomes a test.

Agents4PLC: the loop, automated



only code that compiles AND passes verification leaves the loop

Agents4PLC = propose, check, repair, with a compiler and a verifier as the two checkers.

Agents4PLC: what the verification gate buys

GPT-4o, Table I	easy (16 tasks)	medium (7 tasks)
Agents4PLC: compiles	100 %	100 %
Agents4PLC: verifiable	**68.8 %**	**42.9 %**
LLM4PLC (earlier pipeline): verifiable	**12.5 %**	**0 %**

verifiable = the generated code passes formal verification of the task's properties, not just compilation

Lesson: the verifier gate catches what the LLM misses, but not every task ends verified. Unverified code must not be deployed, and the specifications need an engineer's review.

Liu, Zeng, Wang et al., Agents4PLC, arXiv 2410.14209 (2024); LLM4PLC: Fakih et al., ICSE-SEIP 2024

Verification turns "the LLM says it works" into "the checker says it holds", where it can.

Who does what in verified PLC development



engineer

requirements → specification
reviews LLM-drafted specs
signs off



LLM agents

write and repair ST code
draft specifications
explain counterexamples



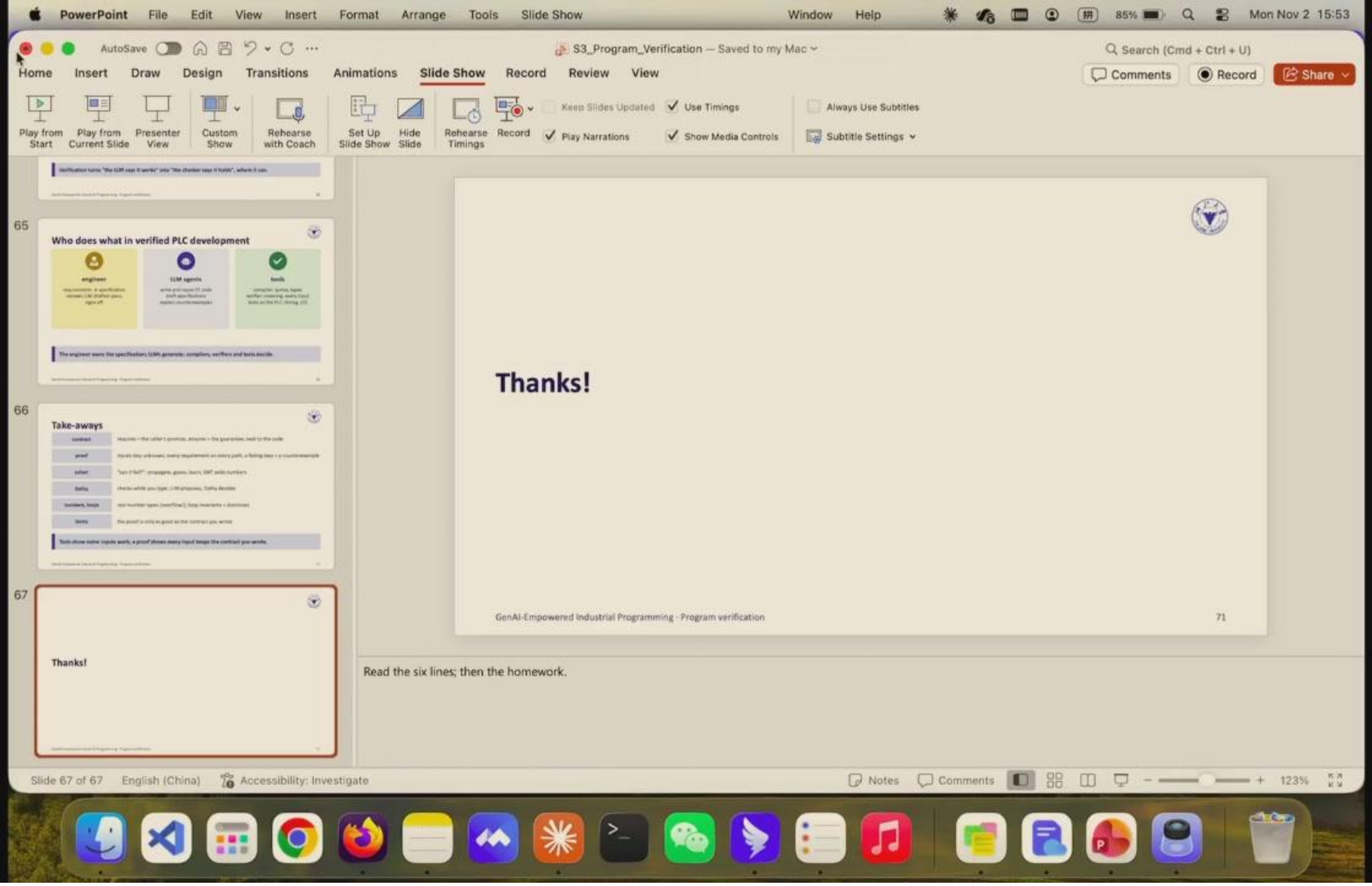
tools

compiler: syntax, types
verifier: meaning, every input
tests on the PLC: timing, I/O

The engineer owns the specification; LLMs generate; compilers, verifiers and tests decide.



Thanks!


PowerPoint File Edit View Insert Format Arrange Tools Slide Show Window Help
Mon Nov 2 15:53
S3_Program_Verification — Saved to my Mac
Home Insert Draw Design Transitions Animations Slide Show Record Review View
Comments Record Share
Play from Start
Play from Current Slide
Presenter View
Custom Show
Rehearse with Coach
Set Up Slide Show
Hide Slide
Rehearse Timings
Record
Keep Slides Updated
Play Narrations
Use Timings
Show Media Controls
Always Use Subtitles
Subtitle Settings
65
Who does what in verified PLC development
66
Take-aways
67
Thanks!
Thanks!
GenAI-Empowered Industrial Programming · Program verification
71
Read the six lines; then the homework.
Slide 67 of 67
English (China)
Accessibility: Investigate
Notes
Comments
123%


USB-C
IN : NO SIGNAL
OUT : 3840X2160P 60


No Signal



24.9cm X 25.4cm
冊絹设色
台北故宫博物院藏

《苹婆山鸟图》
黄筌 · 宋代

15:53
2026/11/2